



33. Python: PyGame

Create games, animations, and interactive worlds with Python.

Previous Sections:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 32. Python: Matplotlib](#)

[!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) 32.1. Python: Seaborn](#)

Contents:

[1. Introduction to PyGame](#)

[2. Basic Operations on PyGame](#)

[2.1. Drawing Shapes](#)

[2.2. Event Handling](#)

[2.3. Working with Text](#)

[2.4. Working with Images](#)

[2.5. Buttons](#)

[2.6. Winning Conditions](#)

[3. Creating Simple Games](#)

[3.1. Snake Game](#)

[3.2. Connect Four \(2-Player\)](#)

[Summary](#)

[References](#)

Imagine creating your own video game using Python. A character that moves across the screen. A ball that bounces. A spaceship fighting enemies. A simple platformer. These are all possible with **PyGame**.

PyGame is one of the most popular libraries for game development in Python. It provides tools for creating windows, drawing graphics, handling keyboard and

mouse input, playing sounds, and managing game logic.

While professional games are often built using large game engines, PyGame is an excellent starting point for learning how games work behind the scenes. It is simple enough for beginners, yet powerful enough to create surprisingly complex projects.

In this chapter, we will focus on the fundamentals of PyGame—the core concepts that every game developer should understand before building larger projects.

1. Introduction to PyGame

Before we create games, we need a game window. Every PyGame program starts by initializing the library, creating a display window, and running a game loop. The game loop continuously checks for events and updates the game while it is running.

Let's begin with the simplest possible PyGame application:

```
import pygame

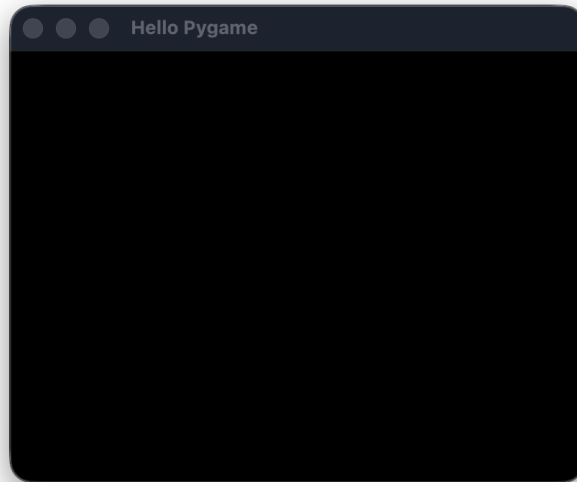
# Initialize Pygame
pygame.init()

# Set up the game window
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("Hello Pygame")

# Game loop
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

# Quit Pygame
pygame.quit()
```



This program does not draw anything yet. It simply creates a window and keeps it open until the user closes it. Although the code looks simple, it introduces several of the most important concepts in PyGame:

- `pygame.init()` : Initializes all PyGame modules and prepares the library for use.
- `pygame.display.set_mode()` : Creates the game window. The tuple `(400, 300)` specifies the width and height of the window in pixels.
- `pygame.display.set_caption()` : Sets the title shown at the top of the window.
- `pygame.event.get()` : Used for handling events, user interactions such as keyboard input, mouse clicks, window events
- `while running:` : The game loop is the heart of every PyGame application. As long as `running` remains `True`, the game continues to execute. Most PyGame programs spend almost their entire lifetime inside this loop.
- `pygame.QUIT` : When the user clicks the window's close button, PyGame generates a `QUIT` event. At which `running = False`. This tells the program to stop the game loop and exit gracefully.
- `pygame.quit()` : Shuts down PyGame and releases any resources it was using. Without it, the program may not close properly on some systems.

This may look like a lot of code just to open a window, but nearly every PyGame project follows this same structure:

Initialize PyGame → Create a game window → Run the game loop → Process events → Exit cleanly

Once you understand these five steps, you're ready to start drawing graphics, handling user input, and building interactive games.

2. Basic Operations on PyGame

Now that we know how to create a PyGame window, it's time to make it do something interesting. Most games are built from a few fundamental components:

- Drawing objects
- Handling user input
- Displaying text
- Showing images
- Creating buttons and interactions
- Defining objectives and winning conditions

Let's explore each of these one step at a time.

2.1. Drawing Shapes

Before we create characters and animations, we first need to draw something on the screen. PyGame provides several built-in functions for drawing simple shapes: Rectangles; Circles; Lines; Polygons

```
# Drawing Shapes
import pygame

pygame.init()

screen = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Drawing Shapes")

running = True

while running:
    screen.fill((255, 255, 255))
```

```

# Rectangle
pygame.draw.rect(screen, (255, 0, 0), (50, 50, 100, 60))

# Circle
pygame.draw.circle(screen, (0, 0, 255), (300, 200), 50)
# Line
pygame.draw.line(screen, (0, 255, 0), (0, 0), (600, 400), 5)

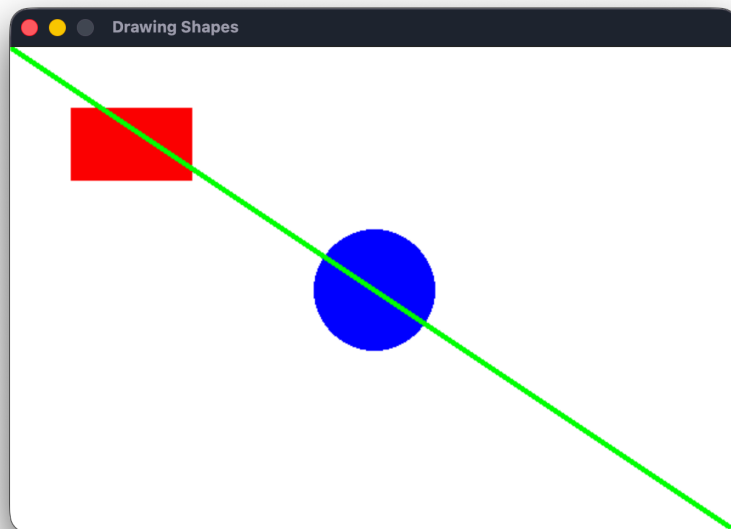
pygame.display.update()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

pygame.quit()

```

In this example: A red rectangle; a blue circle; and a green diagonal line appear on the screen.



Think of these shapes as the building blocks of a game. Many prototypes begin with simple rectangles before replacing them with actual characters and sprites.

2.2. Event Handling

Drawing a shape is nice. But games become interesting when objects respond to player actions. Let's move a square using the arrow keys.

```
# Event Handling
import pygame

pygame.init()

screen = pygame.display.set_mode((600, 400))

x = 250
y = 150
speed = 5

running = True

while running:
    screen.fill((255, 255, 255))

    pygame.draw.rect(screen, (255, 0, 0), (x, y, 50, 50))
    keys = pygame.key.get_pressed()

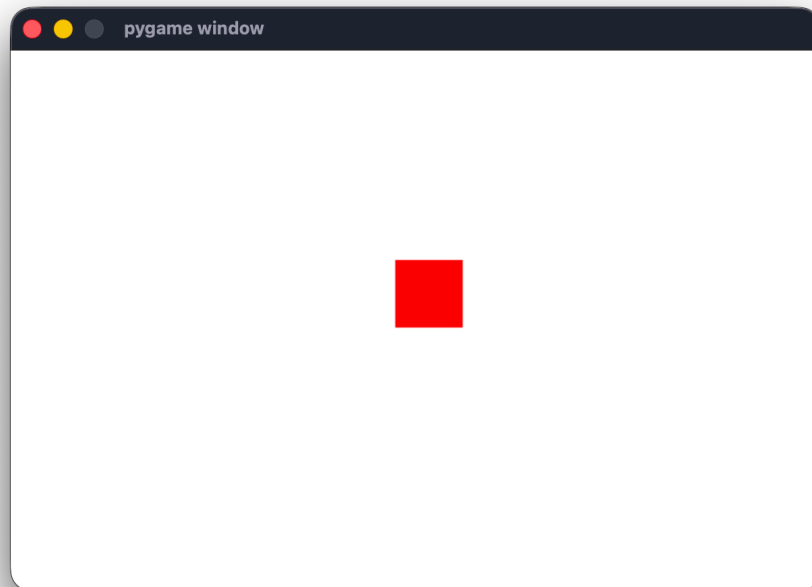
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        x -= speed
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        x += speed
    if keys[pygame.K_UP] or keys[pygame.K_w]:
        y -= speed
    if keys[pygame.K_DOWN] or keys[pygame.K_s]:
        y += speed

    pygame.display.update()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```
running = False

pygame.quit()
```



Now the square can move in four directions: Left, Right, Up, Down using either the arrows or **W, A, S, D**. This idea forms the basis of player movement in most games.

2.3. Working with Text

Games often need to display information: Score; Health; Instructions; Dialogue. PyGame uses fonts to render text.

```
# Working with Text
import pygame

pygame.init()

screen = pygame.display.set_mode((600, 400))
font = pygame.font.SysFont("Arial", 36)
```

```

running = True

while running:
    screen.fill((255, 255, 255))

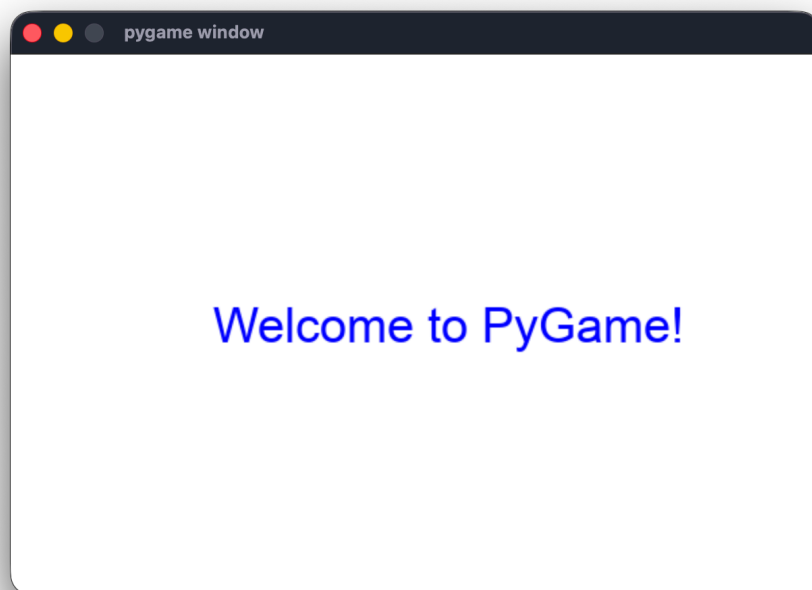
    text = font.render("Welcome to PyGame!", True, (0, 0, 255))
    screen.blit(text, (150, 180))

    pygame.display.update()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

pygame.quit()

```



The process is: **Create a font** → **Render text** → **Display it using `blit()`** . Almost every game uses text somewhere.

2.4. Working with Images

Shapes are useful for learning, but games usually use images. PyGame can load images from files.

player.png

```
# Working with Images
import pygame

pygame.init()
screen = pygame.display.set_mode((600, 400))
player = pygame.image.load("player.png")

running = True

while running:
    screen.fill((255, 255, 255))
    screen.blit(player, (100, 50))

    pygame.display.update()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

pygame.quit()
```



The key method is: `pygame.image.load()`, which loads an image file into memory. And: `screen.blit()`, which draws the image onto the screen.

You can replace `"player.png"` with any image of your choice.

2.5. Buttons

Many games use buttons for: Menus; Start screens; Settings; Interactions Let's create a button that makes a square jump.

```
import pygame

pygame.init()

screen = pygame.display.set_mode((600, 400))

x = 250
y = 250

button = pygame.Rect(230, 50, 140, 50)

running = True
```

```

while running:
    screen.fill((255, 255, 255))

    pygame.draw.rect(screen, (255, 0, 0), (x, y, 50, 50))
    pygame.draw.rect(screen, (0, 200, 0), button)

    font = pygame.font.SysFont(None, 30)

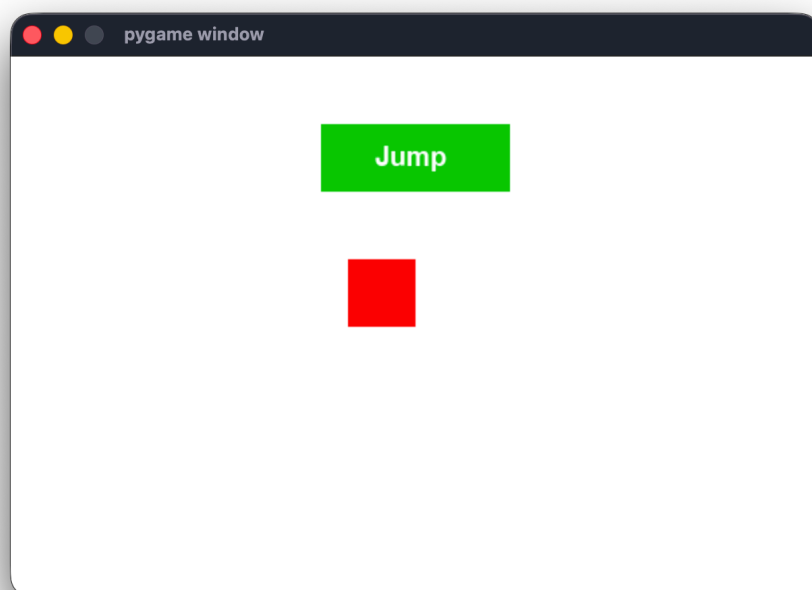
    text = font.render("Jump", True, (255, 255, 255))
    screen.blit(text, (270, 65))

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.MOUSEBUTTONDOWN:
            if button.collidepoint(event.pos):
                y -= 100

    pygame.display.update()

pygame.quit()

```



When the button is clicked: `button.collidepoint(event.pos)` , checks whether the mouse click occurred inside the button. If it did, the square jumps upward. This same idea can be used to create: Start buttons; Exit buttons; Shop menus; Character selection screens.

2.6. Winning Conditions

Every game needs a goal. One common approach is collision detection. Suppose the player must reach a treasure chest.

```
import pygame

pygame.init()

screen = pygame.display.set_mode((600, 400))

player = pygame.Rect(50, 50, 50, 50)
goal = pygame.Rect(500, 300, 60, 60)

font = pygame.font.SysFont(None, 50)

won = False
running = True

while running:
    screen.fill((255, 255, 255))

    keys = pygame.key.get_pressed()

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        player.x -= 5

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        player.x += 5

    if keys[pygame.K_UP] or keys[pygame.K_w]:
        player.y -= 5
```

```

if keys[pygame.K_DOWN] or keys[pygame.K_s]:
    player.y += 5

pygame.draw.rect(screen, (0, 0, 255), player)
pygame.draw.rect(screen, (255, 215, 0), goal)

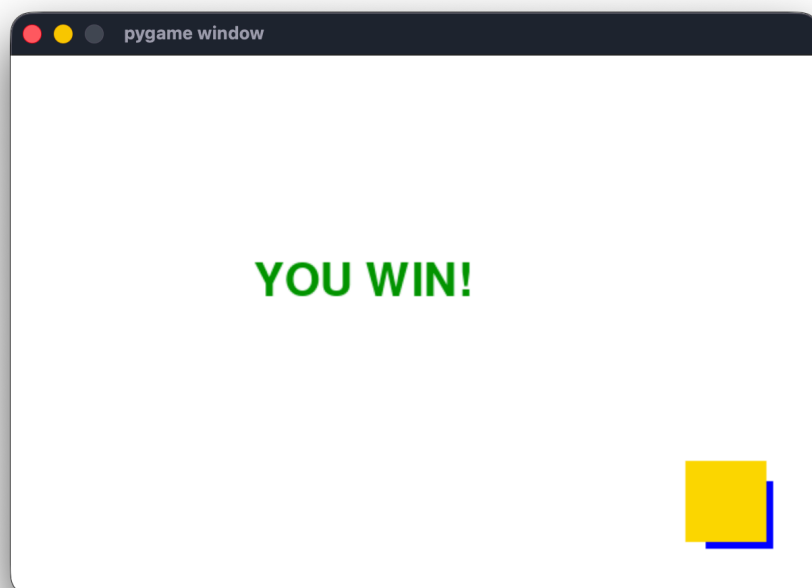
if player.colliderect(goal):
    won = True
if won:
    text = font.render("YOU WIN!", True, (0, 150, 0))
    screen.blit(text, (180, 150))

pygame.display.update()

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

pygame.quit()

```



The important method is: `player.collidect(goal)` , which checks whether two rectangles overlap. When the player reaches the goal, the game displays: `YOU WIN!` .

This simple idea can be expanded into collecting coins, reaching checkpoints, defeating enemies, solving puzzles, completing levels

These above concepts form the foundation of nearly every 2D game. Once you master them, you can begin combining them into larger projects such as platformers, RPGs, puzzle games, shooters, and simulations.

3. Creating Simple Games

So far, we have learned how to draw shapes, handle events, display text, load images, and detect collisions. Now it is time to combine these concepts into complete games.

In this section, we will build two classic projects:

- Snake Game
- Connect Four (2-Player Mode)

These examples demonstrate how the concepts from the previous section work together to create interactive games.

3.1. Snake Game

Snake is one of the most famous arcade games ever created.

The idea is simple:

- Control a snake using the keyboard.
- Eat food to grow longer.
- Avoid hitting the walls.
- Avoid colliding with yourself.

Every time the snake eats food, the score increases and the snake becomes longer.

- Controls

Key	Action
↑ or W	Move Up
↓ or S	Move Down
← or A	Move Left
→ or D	Move Right

```

import pygame
import random

pygame.init()

# Window Setup
WIDTH = 600
HEIGHT = 400
BLOCK = 20
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake Game")
clock = pygame.time.Clock()

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GREEN = (0, 255, 0)
RED = (255, 0, 0)

# Fonts
font = pygame.font.SysFont("Arial", 25)
score_font = pygame.font.SysFont("Arial", 20)

def run_game():
    # Game variables reset
    x = WIDTH // 2
    y = HEIGHT // 2
    dx = BLOCK
    dy = 0
    snake = []

```

```

length = 1

food_x = random.randrange(0, WIDTH, BLOCK)
food_y = random.randrange(0, HEIGHT, BLOCK)

game_active = True
game_over = False

while game_active:

    # Game over loop
    while game_over:
        screen.fill(WHITE)
        text = font.render("Game Over! Press C to Play
Again "
                                "or Q to Quit", True, RED)
        screen.blit(text, [WIDTH // 12, HEIGHT // 3])
        pygame.display.update()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False # Exit completely
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_q:
                return False # Quit the game
            if event.key == pygame.K_c:
                return True # Restart the game loop

# Event Handling loop
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        game_active = False
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_q:
            game_active = False
        elif event.key in [pygame.K_LEFT, pygame.K_
a]:

```



```

        dx = -BLOCK
        dy = 0
    elif event.key in [pygame.K_RIGHT, pygame.K_
_d]:
        dx = BLOCK
        dy = 0
    elif event.key in [pygame.K_UP, pygame.K_
w]:
        dy = -BLOCK
        dx = 0
    elif event.key in [pygame.K_DOWN, pygame.K_
s]:
        dy = BLOCK
        dx = 0

# Movement updates
x += dx
y += dy

# Wall Collision Checks
if x < 0 or x >= WIDTH or y < 0 or y >= HEIGHT:
    print("Game Over!!")
    game_over = True

screen.fill(WHITE)

# Draw Food
pygame.draw.rect(screen, GREEN, (food_x, food_y, BL
OCK, BLOCK))

# Snake growth tracking
head = [x, y]
snake.append(head)
if len(snake) > length:
    snake.pop(0)

# Self-Collision Checks
for part in snake[:-1]:

```

```

        if part == head:
            print("Game Over!!")
            game_over = True

    # Draw Snake
    for part in snake:
        pygame.draw.rect(screen, BLACK, (part[0], part
[1], BLOCK, BLOCK))

    # Food eating check
    if x == food_x and y == food_y:
        food_x = random.randrange(0, WIDTH, BLOCK)
        food_y = random.randrange(0, HEIGHT, BLOCK)
        length += 1

    # Draw Score in Top Left
    score = length - 1
    score_text = score_font.render(f"Score: {score}", T
rue, BLACK)
    screen.blit(score_text, [10, 10])

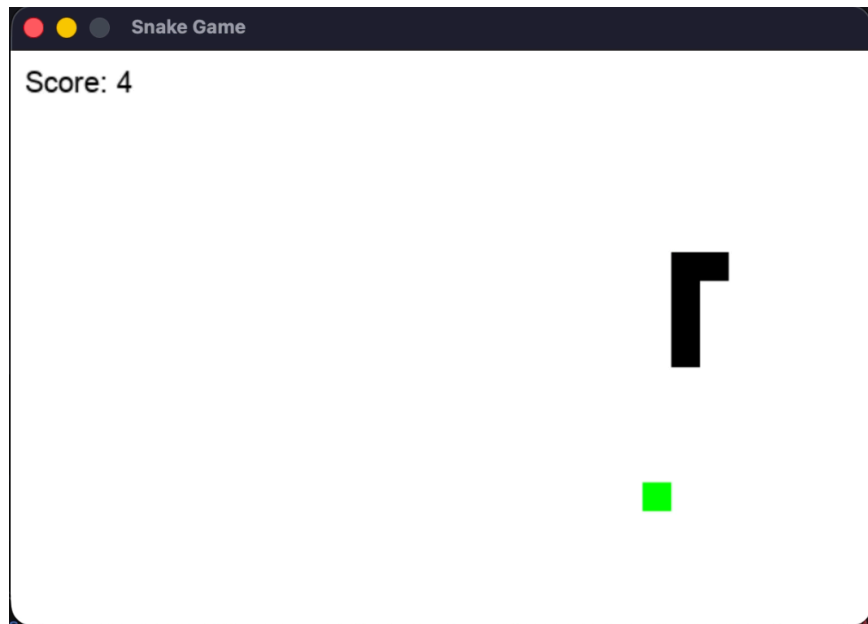
    pygame.display.update()
    clock.tick(10)

    return False

# Main Execution Loop
running = True
while running:
    running = run_game()

pygame.quit()

```



This game demonstrates: Movement; Keyboard input; Collision detection; Random food generation; Score mechanics through snake growth; Display Game Over; Quit the game by pressing a key.

3.2. Connect Four (2-Player)

Connect Four is a strategy game played by two players. The goal is to connect four pieces: Horizontally; Vertically; Diagonally. Players take turns dropping pieces into columns.

Unlike some versions, this implementation is **purely 2-player** and starts immediately when the program runs.

- Controls
 - Click a column with the mouse.
 - The piece will fall into that column.
 - Players alternate turns automatically.

```
import pygame
import numpy as np

pygame.init()
```

```

ROWS = 6
COLS = 7
SIZE = 100

BLUE = (0, 0, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
YELLOW = (255, 255, 0)
WHITE = (255, 255, 255)

font = pygame.font.SysFont("Arial", 40, bold=True)
sub_font = pygame.font.SysFont("Arial", 20)

board = np.zeros((ROWS, COLS))
screen = pygame.display.set_mode((COLS * SIZE, (ROWS + 1) *
SIZE))
pygame.display.set_caption("Connect Four")
turn = 0

def drop_piece(row, col, piece):
    board[row][col] = piece

def valid_location(col):
    return board[ROWS - 1][col] == 0

def next_row(col):
    for r in range(ROWS):
        if board[r][col] == 0:
            return r
    return None

def check_tie():
    return np.all(board != 0)

```

```

def winning(piece):
    # Horizontal
    for c in range(COLS - 3):
        for r in range(ROWS):
            if (board[r][c] == piece and board[r][c + 1] ==
piece
                                and board[r][c + 2] == piece and board
[r][c + 3] == piece):
                return True

    # Vertical
    for c in range(COLS):
        for r in range(ROWS - 3):
            if (board[r][c] == piece and board[r + 1][c] ==
piece
                                and board[r + 2][c] == piece and board
[r + 3][c] == piece):
                return True

    for c in range(COLS - 3):
        for r in range(ROWS - 3):
            if (board[r][c] == piece and board[r + 1][c +
1] == piece
                                and board[r + 2][c + 2] == piece and bo
ard[r + 3][c + 3] == piece):
                return True

    # Diagonal
    for c in range(COLS - 3):
        for r in range(3, ROWS):
            if (board[r][c] == piece and board[r - 1][c +
1] == piece
                                and board[r - 2][c + 2] == piece and bo
ard[r - 3][c + 3] == piece):
                return True
    return False

```

```

def draw_board():
    for c in range(COLS):
        for r in range(ROWS):
            pygame.draw.rect(screen, BLUE, (c * SIZE, (r +
1) * SIZE, SIZE, SIZE))
            pygame.draw.circle(screen, BLACK, (c * SIZE + S
SIZE // 2,
                                                    (r + 1) * SI
ZE + SIZE // 2), 40)
        for c in range(COLS):
            for r in range(ROWS):
                if board[r][c] == 1:
                    pygame.draw.circle(screen, RED, (c * SIZE +
SIZE // 2,
                                                    (ROWS - r)
* SIZE + SIZE // 2), 40)
                elif board[r][c] == 2:
                    pygame.draw.circle(screen, YELLOW, (c * SIZ
E + SIZE // 2,
                                                    (ROWS -
r) * SIZE + SIZE // 2), 40)
            pygame.display.update()

def reset_game():
    global board, turn
    board = np.zeros((ROWS, COLS))
    turn = 0
    pygame.draw.rect(screen, BLACK, (0, 0, COLS * SIZE, SIZ
E))
    draw_board()

draw_board()
running = True
game_over = False

```

```

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_q:
                running = False
            if event.key == pygame.K_r:
                reset_game()
                game_over = False

        if event.type == pygame.MOUSEMOTION and not game_over:
            pygame.draw.rect(screen, BLACK, (0, 0, COLS * SIZE, SIZE))
            pos_x = event.pos[0]
            current_color = RED if turn == 0 else YELLOW
            pygame.draw.circle(screen, current_color, (pos_x, SIZE // 2), 40)
            pygame.display.update()

        if event.type == pygame.MOUSEBUTTONDOWN and not game_over:
            col = event.pos[0] // SIZE
            if valid_location(col):
                row = next_row(col)
                piece = 1 if turn == 0 else 2
                drop_piece(row, col, piece)

                if winning(piece):
                    game_over = True
                    text_color = RED if piece == 1 else YELLOW

                    winner_name = "Player 1 (Red)" if piece == 1 else "Player 2 (Yellow)"
                    pygame.draw.rect(screen, BLACK, (0, 0,

```

```

COLS * SIZE, SIZE))
        win_text = font.render(f"{winner_name}
Wins!", True, text_color)
        sub_text = sub_font.render("Press R to
Restart or Q to Quit", True, WHITE)
        screen.blit(win_text, (20, 15))
        screen.blit(sub_text, (20, 65))
    elif check_tie():
        game_over = True
        pygame.draw.rect(screen, BLACK, (0, 0,
COLS * SIZE, SIZE))
        tie_text = font.render("It's a Tie!", T
rue, WHITE)
        sub_text = sub_font.render("Press R to
Restart or Q to Quit", True, WHITE)
        screen.blit(tie_text, (20, 15))
        screen.blit(sub_text, (20, 65))
    else:
        turn = (turn + 1) % 2

    draw_board()

pygame.quit()

```




This project demonstrates: Mouse interaction; Grid-based games; Turn-based gameplay; Win detection algorithms; Board management; Announcing Winners.

Summary

This section covered some basic methods, and two complete PyGame projects that combine many of the concepts introduced throughout.

Using PyGame, we can now:

- Move game objects with the keyboard
- Detect collisions
- Create game loops
- Generate random objects
- Handle mouse input
- Build turn-based games
- Detect winning conditions

These projects may be simple, but they introduce many of the same ideas used in larger games. With these foundations, you can begin creating your own

platformers, puzzle games, arcade games, and multiplayer experiences.

References

<https://www.pygame.org/wiki/tutorials>

<https://www.geeksforgeeks.org/python/pygame-tutorial/>

Next Section:

[`</>` 34. Python: Audio Processing](#)